



عملي

7

12

1440 sp

كلية الهندسة المعلوماتية

السنة الثالثة

3D models + Effects



عمل طلابي

محتوى مجاني غير مخصص للبيع التجاري

1/12/2024

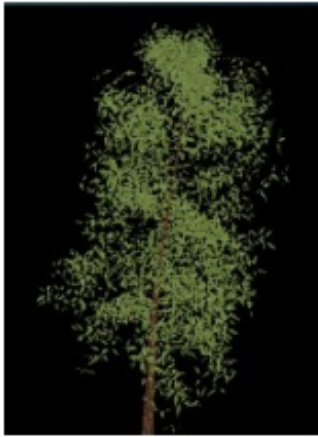
بيانات حاسوبية

RB Informatics;

3D models

ما هي الـ 3D models؟

هي مجسمات ثلاثية الأبعاد جاهزة ومرسومة باستخدام بعض البرامج مثل Autodesk 3ds max و Autodesk maya. يمكننا إضافتها إلى مشهدين بدلاً من رسمها يدوياً باستخدام الكود كما أنها تجعل المشهد أكثر واقعية. أمثلة عن الـ 3D models: مباني، شخصيات، أشجار، سيارات، ...



خطوات إضافة نموذج ثلاثي الأبعاد 3D model:

1. إضافة المكتبات إلى المشروع:

يجب تضمين المكتبات اللازمة والتي تدعم قراءة وتحميل الـ 3D models في المشروع، ويمكنكم الحصول على المكتبات عن طريق مسح الـ QR code التالي:

بعد تنزيل الملفات، يجب وضع جميع الملفات التي تنتهي باللاحقة .h في المسار التالي من أجل الـ visual studio 2012:

"C:\Program Files (x86)\Microsoft Visual Studio 11.0\VC\include"

أو في المسار التالي من أجل الـ clion:

"C:\Program Files\JetBrains\CLion 2022.1.1\bin\mingw\x86_64-w64-mingw32\include"

أو يمكنكم تضمين الملفات في المشروع كما شرحنا في الملحق.



ثم يجب وضع جميع الملفات التي تنتهي باللاحقة .cpp في مجلد الـ src الخاص بالمشروع، ثم تضمين ملف المكتبة في ملف الـ main الأساسي، مثلاً:

```
#include "Model_3DS.h"
```

وهكذا لباقي الملفات.

2. تعريف المتغير الخاص بالـ 3D model قبل التابع InitGL:

يجب تعريف متغير يشير إلى الـ 3D model الذي سنستخدمه، حيث سيكون هذا المتغير عبارة عن pointer من نوع Model_3DS.

ملاحظة: يوجد في الـ QR code الموجود أعلاه نموذج الـ tree الذي سنطبق عليه مثال الكود ويفضل وضعه في مجلد المشروع لتجنب استخدام المسار الكامل له لاحقاً.

مثال:

```
Model_3DS *tree; // تعريف مؤشر للنموذج ثلاثي الأبعاد tree
```

3. تحميل النموذج داخل الـ InitGL:

أي إسناد الـ 3D model الذي لدينا (Tree.3ds) إلى المتغير tree الذي عرفناه في الخطوة السابقة.

```
tree = new Model_3DS(); // إنشاء object من نوع Model_3DS
tree -> Load("Tree.3ds"); // تحميل النموذج ثلاثي الأبعاد من ملف Tree.3ds
```

4. ربط الـ Textures مع الـ 3D model:

عادة يكون للنموذج ثلاثي الأبعاد أكثر من جزء، ويجب ربط textures مختلفة لكل جزء منه، أي تعيين صور textures معينة للأجزاء المختلفة للنموذج، فمثلاً في مثال الشجرة يوجد لدينا قسمان: bark للجذع وقسم للأوراق leaf. **أولاً:** يجب تحميل الـ textures المناسبة داخل الـ InitGL:

```
GLTexture bark, leaf; // تعريف متغيرين للـ Textures
leaf.LoadBMP("leaf.bmp"); // تحميل الصورة leaf.bmp
bark.LoadBMP("bark.bmp"); // تحميل الصورة bark.bmp
```

ثانياً: ربط الـ Textures مع الأجزاء المختلفة للنموذج:

```
tree -> Materials[0].tex = bark;
tree -> Materials[1].tex = bark;
tree -> Materials[2].tex = leaf;
tree -> Materials[3].tex = leaf;
tree -> Materials[4].tex = leaf;
tree -> Materials[5].tex = leaf;
tree -> Materials[6].tex = leaf;
```



ملاحظة: النموذج tree يحتوي على 7 أجزاء، أول جزئين يعبران عن الجذع وبقية الأجزاء تعبر عن الأوراق، ويختلف معنى الأجزاء وعددها من نموذج إلى آخر ولذلك يعتمد هذا القسم على التجريب لمعرفة عدد الأجزاء ومعرفة ماذا يمثل كل جزء من النموذج.

5. التحكم في خصائص النموذج:

يجب تحديد إحداثيات النموذج (pos) إضافة إلى حجمه (scale)، ويتم ذلك إما في الـ InitGL في حال كان موقعه ثابت دوماً في المشهد، أو في DrawGLScene في حال وجود حركة ستطبق عليه.

```
tree -> pos.x = 0; // تحديد موقع النموذج على المحور x
tree -> pos.y = -1.5; // تحديد موقع النموذج على المحور y
tree -> pos.z = -5; // تحديد موقع النموذج على المحور z
tree -> scale = 0.2; // تحديد نسبة حجم النموذج لتكون 0.2 من الحجم الأصلي
```

6. رسم النموذج داخل التابع DrawGLScene:

بعد تطبيق جميع الخطوات السابقة، يمكننا رسم الـ 3D Model باستخدام الدالة draw:

```
tree -> Draw();
```

ويكون شكل الخرج النهائي:



ملاحظات:

- كثرة استخدام النماذج ثلاثية الأبعاد الثقيلة قد تؤدي إلى بطء في أداء البرنامج، ولذلك يفضل تقليل عدد النماذج أو استخدام نماذج خفيفة لا يزيد حجمها عن الـ 4 ميغابايت.
- إليك بعض المواقع التي يمكنكم تحميل الـ 3D models منها مجاناً:

Blender.org / Free3D.com / Archive3D.org

Shadow

مقدمة:

يوجد في OpenGL العديد من الـ Buffers لتخزين مختلف المعلومات وستحدث عن اثنين من أهم الـ Buffers:

Color buffer:

يخزن اللون لكل بيكسل في المشهد حيث يتم تحديد اللون باستخدام التتابع المخصصة لتغييره ويطبق على الأشكال التي ترسم لاحقاً.

Depth Buffer:

يخزن معلومات العمق لكل بيكسل، مما يسمح بتحديد أي الأجسام يجب أن يظهر في المقدمة وأيها في الخلف. تستخدم خاصية Depth Test لمقارنة العمق وتكون مفعلة بشكل افتراضي، ولكن يمكن تعطيلها عند الحاجة لرسم الأشكال فوق بعضها أو لرسم الظلال.

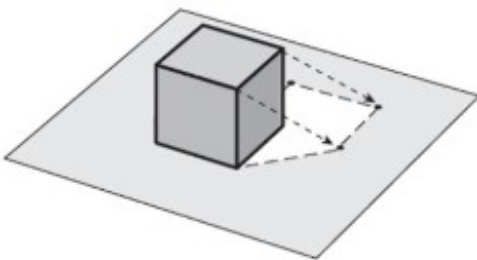
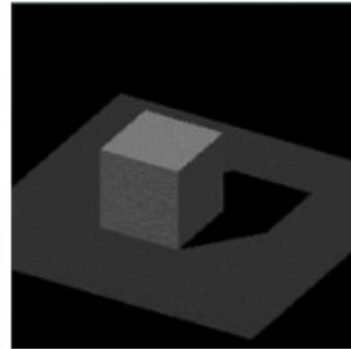
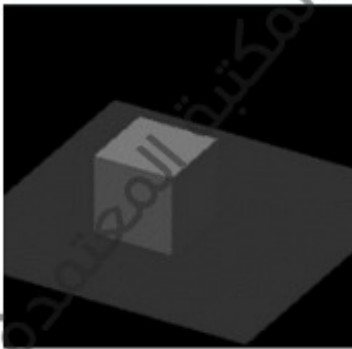
الظلال Shadows في OpenGL:

تضيف الظلال عمقاً وواقعية إلى المشاهد ثلاثية الأبعاد، حيث تظهر التأثيرات الناتجة عن مصادر الضوء وكيفية تأثر الأجسام بوجود بعضها البعض.

ما هي الظلال في OpenGL؟

من الناحية النظرية: الظل يتكون عندما يحجب جسم ما الضوء عن جسم آخر خلفه، ويتم تمثيل المناطق التي لا يصلها الضوء بالظلال.

في OpenGL: يمكن رسم الظل عن طريق تسطيح الشكل الأصلي على السطح الذي يقع عليه الظل، ثم رسمه باللون الأسود أو بلون داكن.



تتمحور آلية رسم الظل حول خطوة أساسية وهي تسطيح الجسم على السطح الذي يقع عليه باستخدام بعض التعديلات على المصفوفات، وبعد ذلك رسم الشكل المُسطح (الظل) بلون داكن.

خطوات رسم الظلال في OpenGL:

1. flattening the object (تسطيح الجسم):

يجب أن نقوم بتعديل مصفوفة العرض Modelview Matrix بحيث تكون جميع الأجسام التي ترسم في هذه المصفوفة الآن مسطحة على السطح الذي سيقع عليه الظل.

2. plane position (تحديد موضع السطح):

يجب معرفة موقع السطح الذي سيقع عليه الظل، لأنه المكان الذي سيُرسم عليه الجسم المسطح. للقيام بالخطوتين 1 و 2 يجب معرفة معادلة هذا السطح. المعادلة القياسية للسطح في الفضاء ثلاثي الأبعاد:

$$Ax + By + Cz + D = 0$$

حيث (A, B, C) هي اتجاهات السطح و (D) هو الإزاحة.

ملاحظة: لا يهمنا كيف يتم حساب المعادلات والمصفوفات لأنها تحسب تلقائياً باستخدام توابع جاهزة في المكاتب، ويمكنكم الوصول إلى الملفات اللازمة لاستخدام الظلال من الـ QR code الموجود في بداية المحاضرة، وهنا سنستخدم مكتبة Math3D لحساب الظل.

لحساب معادلة السطح، نستخدم التابع ذو الشكل العام:

```
M3dGetPlaneEquation(M3DVector 4f vPlaneEquation, points[0], points[1], points[2]);
```

حيث:

vPlaneEquation هو الذي سيحوي معادلة السطح بعد استدعاء التابع.

points هو فيكتور يحوي ثلاث نقاط من المستوي ليست على استقامة واحدة، ويتم تعريفه بالشكل التالي:

```
M3DVector3f points[3] = {{x1, y1, z1}, {x2, y2, z2}, {x3, y3, z3}};
```

3. اتجاه ومسافة مصدر الضوء:

اتجاه مصدر الضوء هو ما يحدد شكل وحجم الظل، فعندما يكون الضوء قريباً أو بعيداً يتغير حجم الظل، وعندما يتحرك مصدر الضوء يتغير شكل الظل.

لحساب مصفوفة الظل بناءً على معادلة السطح وموقع الضوء نستخدم التابع ذو الشكل العام:

```
m3dMakePlanarShadowMatrix(M3DMatrix44f shadowMat, M3DVector4f vPlaneEquation, GLfloat lightPos[4]);
```

حيث:

shadowMat هو الذي سيحوي مصفوفة الظل بعد استدعاء التابع.

vPlaneEquation هو معادلة السطح التي نتجت عن استدعاء التابع في الخطوة السابقة.

lightPos هي المصفوفة المعبرة عن موقع الضوء والتي شرحنا عنها في محاضرة الإضاءة.



مثال كامل لرسم الظلال في OpenGL:

1. تعريف المتغيرات قبل الدالة initGL:

خصائص الإضاءة:

```
GLfloat lightPos[] = { -1.0f, 3.0f, 0.0f, 0.0f }; // موقع الضوء
GLfloat LightAmb[] = { 1.0f, 1.0f, 1.0f, 1.0f }; // الإضاءة المحيطة
GLfloat LightDiff[] = { 0.6f, 0.6f, 0.6f, 1.0f }; // الإضاءة المنتشرة
```

معادلة السطح ومصفوفة الظل:

```
M3DMatrix44f shadowMat; // مصفوفة رسم الظل
M3DVector4f vPlaneEquation; // معادلة السطح
```

2. إعداد الظل والإضاءة في initGL:

تحديد ثلاث نقاط على السطح الذي سيقع عليه الظل (ليست على استقامة واحدة):

```
M3DVector3f points[3] = {{-30.0f, -2.0f, -20.0f},
{-30.0f, -2.0f, 20.0f}, {40.0f, -2.0f, 20.0f}};
```

تفعيل اختبار العمق وإعداد الإضاءة:

```
glEnable(GL_DEPTH_TEST); // تفعيل اختبار العمق
glEnable(GL_LIGHTING); // تفعيل الإضاءة
glLightfv(GL_LIGHT1, GL_POSITION, lightPos);
glLightfv(GL_LIGHT1, GL_AMBIENT, LightAmb);
glLightfv(GL_LIGHT1, GL_DIFFUSE, LightDiff);
glEnable(GL_LIGHT1); // تفعيل الضوء الأول
```

حساب معادلة السطح ومصفوفة الظل:

```
m3dGetPlaneEquation(vPlaneEquation, points[0], points[1],
points[2]);
// حساب معادلة السطح
m3dMakePlanarShadowMatrix(shadowMat, vPlaneEquation, lightPos);
// حساب مصفوفة الظل
```

3. رسم المشهد والظل في DrawGLScene:

رسم الأرضية:

```
gluLookAt(0, 0, 10, 0, 0, 0, 0, 1, 0);
glBegin(GL_QUADS); // رسم الأرضية
glColor3f(0, 0.2, 0); // اللون الأخضر الداكن
glVertex3f(40.0f, -2.0f, -40.0f);
glVertex3f(-40.0f, -2.0f, -40.0f);
glColor3f(0, 1, 0); // لون أخضر فاتح
glVertex3f(-40.0f, -2.0f, 40.0f);
glVertex3f(40.0f, -2.0f, 40.0f);
glEnd();
```

تخصيص شكل مصدر الضوء:

```
glPushMatrix();
glTranslatef(lightPos[0], lightPos[1], lightPos[2]);
glColor3ub(255, 255, 0);
auxSolidSphere(0.1f);
glPopMatrix();
```

رسم الشكل الأصلي:

```
glPushMatrix();
glEnable(GL_LIGHTING); // تفعيل الإضاءة
glRotatef(10, 1.0f, 0.0f, 0.0f); // دوران حول المحور X
glRotatef(45, 0.0f, 1.0f, 0.0f); // دوران حول المحور Y
auxSolidSphere(0.5); // رسم الكرة
glPopMatrix();
```

Stars can't shine
without darkness



```

glDisable(GL_DEPTH_TEST); // تعطيل اختبار العمق للظل
glDisable(GL_LIGHTING); // تعطيل الإضاءة للظل
glPushMatrix();
glMultMatrixf((GLfloat *)shadowMat); // تطبيق مصفوفة الظل
glRotatef(10, 1.0f, 0.0f, 0.0f); // دوران للظل
glRotatef(45, 0.0f, 1.0f, 0.0f); // دوران للظل
glColor3f(0, 0, 0); // اللون الأسود للظل
auxSolidSphere(0.5); // رسم الكرة كظل
glPopMatrix();
glEnable(GL_DEPTH_TEST); // إعادة تفعيل اختبار العمق

```

الخرج:



يمكننا إضافة تحريك للضوء باستخدام مصفوفة الـ keys وسنلاحظ أن الظل يتحرك مع تغيير موضع الإضاءة ليحاكي المشهد في الواقع.

Do something today that
your future self will thank
you for.



Blending

ما هو الـ Blending (المزج)؟

يتيح لنا الـ Blending الجمع بين ألوان الأجسام المرسومة بحيث يظهر بعضها شفافاً أو شبه شفاف، مما يسمح لنا برؤية ما خلفها، ونستفيد منها لرسم شيء زجاجي مثل النوافذ.

عندما يتم تعطيل الـ Blending فإن أي قيمة لون جديدة يتم وضعها في الـ Color Buffer إما أن تستبدل القيم القديمة في حال تعطيل اختبار العمق، أو أن تستبدلها فقط إذا كان الجسم الجديد أقرب إلى الكاميرا في حال تفعيل اختبار العمق. أما عندما يتم تفعيل الـ Blending، فيتم دمج اللون الحالي (المصدر) مع اللون المخزن بالفعل (الهدف) في الـ Color Buffer بدلاً من استبداله بالكامل مما يسمح بظهور الألوان المختلطة (مثل تأثيرات الشفافية).
لتفعيل الـ Blending يكفي كتابة:

```
glEnable(GL_BLEND);
```

كيف يتم الـ Blending في OpenGL؟

دمج الألوان:

هناك نوعان من الألوان في عملية الـ Blending:

- اللون المصدر (Source Color): هو اللون الجديد الذي يتم رسمه (لون الجسم الشفاف).
- اللون الهدف (Destination Color): هو اللون المخزن بالفعل في الـ Color Buffer (لون الجسم الواقع خلف الجسم الشفاف).

معادلة المزج:

عندما يتم تفعيل المزج، يتم داخلياً استخدام معادلة المزج التالية لحساب اللون النهائي لكل بيكسل:

$$C_f = (C_s \times S) + (C_d \times D)$$

حيث:

C_f : هو اللون النهائي بعد المزج (Final Color).

C_s : هو لون المصدر، أي لون الجسم الشفاف (Source Color).

C_d : هو لون الهدف، أي لون الجسم الموجود خلف الجسم الشفاف (Destination Color).

S و D : هي معاملات المزج الخاصة بلون المصدر ولون الهدف، والتي يمكن تحديدها باستخدام `glBlendFunc()`.

دالة المزج `glBlendFunc`:

تتيح الدالة `glBlendFunc` التحكم في كيفية دمج ألوان المصدر والهدف ويتم استدعاؤها بالشكل التالي:

```
glBlendFunc(GLenum sfactor, GLenum dfactor);
```

حيث:

`sfactor`: هو المعامل الذي يحدد كيفية تأثير لون المصدر في عملية المزج.

`dfactor`: هو المعامل الذي يحدد كيفية تأثير لون الهدف في عملية المزج.

يتم تحديد هذه المعاملات بناءً على طريقة المزج التي نريدها، وسنذكر في الجدول التالي المعاملات التي يمكننا استخدامها مع الدالة.

المعامل	الوصف	RGB Blend Factor	Alpha Blend Factor
GL_ZERO	لا تأثير، كل القيم مضروبة في 0 (لا يظهر اللون)	0, 0, 0	0
GL_ONE	استخدام اللون كاملاً (كل القيم مضروبة في 1)	1, 1, 1	1
GL_SRC_ALPHA	لون المصدر مضروباً في قيمة Alpha الخاصة بالمصدر	$\alpha_s, \alpha_s, \alpha_s$	α_s
GL_ONE_MINUS_SRC_ALPHA	لون الهدف مضروباً في (1 - Alpha) حيث Alpha هي شفافية المصدر	$1 - \alpha_s, 1 - \alpha_s, 1 - \alpha_s$	$1 - \alpha_s$
GL_DST_ALPHA	لون الهدف مضروباً بقيمة Alpha الخاصة بالهدف	$\alpha_d, \alpha_d, \alpha_d$	α_d
GL_ONE_MINUS_DST_ALPHA	لون المصدر مضروباً في (1 - Alpha) حيث Alpha هي شفافية الهدف	$1 - \alpha_d, 1 - \alpha_d, 1 - \alpha_d$	$1 - \alpha_d$
GL_SRC_COLOR	استخدام اللون الأصلي للمصدر دون تعديل	R_s, G_s, B_s	α_s
GL_ONE_MINUS_SRC_COLOR	استخدام (لون المصدر - 1)	$1 - R_s, 1 - G_s, 1 - B_s$	$1 - \alpha_s$
GL_DST_COLOR	استخدام اللون الأصلي للهدف دون تعديل	R_d, G_d, B_d	α_d
GL_ONE_MINUS_DST_COLOR	استخدام (لون الهدف - 1)	$1 - R_d, 1 - G_d, 1 - B_d$	$1 - \alpha_d$
GL_SRC_ALPHA_SATURATE	معامل خاص يستخدم لإشباع اللون بناءً على قيمة Alpha المصدر والهدف	$\min(\alpha_s, 1 - \alpha_d), \min(\alpha_s, 1 - \alpha_d), \min(\alpha_s, 1 - \alpha_d)$	1

الاستخدام الشائع للمزج:

`glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);`

GL_SRC_ALPHA: يقوم بأخذ لون المصدر وضرب قيم RGB في قيمة Alpha للشفافية.

GL_ONE_MINUS_SRC_ALPHA: يأخذ لون الهدف ويضربه في (1 - Alpha) حيث Alpha هي شفافية الهدف، مما يعطي تأثير الشفافية المطلوب.

كيف يتم تحديد الشفافية؟

نقوم بتحديد قيمة الشفافية باستخدام التابع glColor4f الذي يقبل أربعة معطيات (أحمر، أخضر، أزرق، الشفافية Alpha):
glColor4f(R, G, B, Alpha);



Alpha = 1 تعني أن الجسم غير شفاف.

Alpha = 0 تعني أن الجسم شفاف بالكامل وغير مرئي.

القيم بين الـ 0 والـ 1 تعني نصف شفافية.

مثال لتطبيق الـ Blending في الكود:

سنقوم برسم مشهد في DrawGLScene يحتوي على كرة غير شفافة ومكعب شفاف:

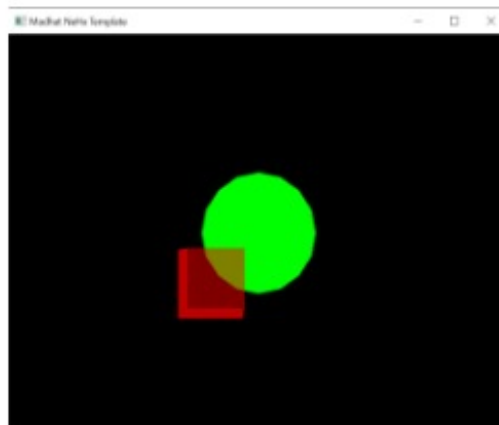
1. رسم الكرة غير الشفافة:

```
glPushMatrix();
glColor4f(0.0f, 1.0f, 0.0f, 1.0f); // اللون الأخضر، بدون شفافية
glTranslatef(0, 0, -4); // نقل الكرة
auxSolidSphere(0.5); // رسم الكرة
glPopMatrix();
```

2. تفعيل الـ Blending ورسم المكعب الشفاف:

```
glPushMatrix();
glEnable(GL_BLEND); // تفعيل المزج
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA); // ضبط دالة المزج

// رسم مكعب شفاف باللون الأحمر (50% شفافية)
glColor4f(1.0f, 0.0f, 0.0f, 0.5f); // اللون الأحمر مع 50% شفافية
glTranslatef(-0.3f, -0.3f, -3.0f); // نقل المكعب
auxSolidCube(0.4f); // رسم المكعب
glDisable(GL_BLEND); // تعطيل المزج بعد رسم المكعب
glPopMatrix();
```



- ترتيب الرسم مهم جداً عند استخدام الشفافية فيجب رسم الأجسام الشفافة بعد الأجسام غير الشفافة لضمان ظهور ما خلف الجسم الشفاف بشكل صحيح، فإذا رُسم الجسم الشفاف أولاً لن يظهر ما خلفه بوضوح.
- الـ Depth Test: يجب الحذر عند استخدام الشفافية مع الـ Depth Test. يمكن تعطيل اختبار العمق مؤقتاً عند رسم الجسم الشفاف لضمان رؤية ما خلفه:

تعطيل اختبار العمق // `glDisable(GL_DEPTH_TEST);`

وهكذا نكون قد أنهينا مقرر عملي البيانيات الحاسوبية لهذا الفصل، نرجو أن تكون الأفكار كافية وشاملة للعمل بالمشروع وأن تكون المحاضرات قد قدمت الفائدة المرجوة منها.

نتمنى لكم دوام التوفيق والنجاح في المشروع.



Sometimes later
Becomes never.
Do it now!